

# 8-WEEK FULL STACK + AI ENGINEER ROADMAP

56-Day Job-Ready Curriculum • MERN + Java/Spring Boot + APIs + Microservices + DevOps + LLM + RAG + Agents + MCP

**Target:** Fresher Software Engineer / Full Stack Developer / Java Backend Developer / AI Engineer / Generative AI Developer

**Core philosophy:** Learn → Code → Build → Test → Deploy → Monitor → Document → Explain in interview.

**Daily commitment:** approximately 6–10 focused hours, with MUST / SHOULD / BONUS priorities.

# 1. Final Skill Map

| Track        | Core skills                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------------------|
| Frontend     | HTML, CSS, JavaScript, TypeScript, React, React Router, Tailwind, Next.js, forms, state, API integration, performance |
| Backend      | Node.js, Express, Java, Spring Boot, REST, authentication, authorization, validation, error handling, testing         |
| Data         | SQL, PostgreSQL/MySQL, MongoDB, Redis, indexing, transactions, caching, aggregation                                   |
| Architecture | API Gateway, microservices, queues, retries, circuit breakers, idempotency, observability                             |
| DevOps       | Linux, Git/GitHub, Docker, Compose, CI/CD, Nginx, cloud deployment, logs, metrics, health checks                      |
| AI           | Python, ML fundamentals, LLMs, prompting, APIs, structured output, embeddings, vector DBs                             |
| GenAI        | RAG, reranking, evaluation, LangChain, LangGraph, agents, tools, MCP, AI security                                     |
| Interview    | DSA, DBMS, OOP, networks, system design, LLD, project explanation, debugging                                          |

# 2. Recommended Technology Stack

| Category   | Primary                                 | Purpose                   |
|------------|-----------------------------------------|---------------------------|
| Frontend   | React + TypeScript + Tailwind           | Production UI             |
| Backend A  | Node.js + Express                       | MERN APIs                 |
| Backend B  | Java + Spring Boot                      | Enterprise backend        |
| Database   | PostgreSQL + MongoDB                    | SQL + NoSQL               |
| Cache      | Redis                                   | Caching, sessions, queues |
| AI backend | Python + FastAPI                        | AI services               |
| AI         | LLM API + embeddings                    | Generative applications   |
| RAG        | Chroma/pgvector + retrieval             | Knowledge systems         |
| Agents     | LangGraph + tools + MCP                 | Agentic applications      |
| DevOps     | GitHub Actions + Docker                 | CI/CD and reproducibility |
| Monitoring | Logs + OpenTelemetry/Prometheus/Grafana | Production observability  |

# 3. 8-Week Overview

| Week | Primary goal                                           | Major deliverable                           |
|------|--------------------------------------------------------|---------------------------------------------|
| 1    | Programming/web foundations + Git + SQL + Python       | Mini full-stack CRUD + SQL/Python exercises |
| 2    | React + Node/Express + MongoDB + Java/Spring Boot      | MERN app + Spring Boot REST API             |
| 3    | Production APIs + auth + Redis + testing               | Authenticated production-style services     |
| 4    | Microservices + payments + Docker + CI/CD + deployment | Sandbox payment/order platform deployed     |
| 5    | AI/ML foundations + LLM + prompting + FastAPI          | LLM application with structured output      |
| 6    | Embeddings + vector DB + RAG + evaluation              | Deployed RAG knowledge assistant            |
| 7    | Agents + LangGraph + tools + MCP + security            | Agentic AI assistant                        |
| 8    | Capstone + system design + monitoring + interviews     | AI e-commerce platform + job portfolio      |

# Week 1 — Foundations

Build strong fundamentals without skipping core concepts.

## Day 1 — Environment + Git + programming baseline

Git/GitHub, terminal, VS Code, HTTP basics; JavaScript variables/functions; Python syntax; DSA arrays; system design client/server; create GitHub repo and baseline README.

## Day 2 — HTML/CSS + Java fundamentals

Semantic HTML, CSS box model, flex/grid; Java types/control flow; Python collections; DSA strings/hash map; Git branching; build responsive profile page.

### **Day 3 — JavaScript deep dive**

ES6, objects, arrays, destructuring, modules; Java methods/OOP intro; Python functions/modules; DSA two pointers; SQL SELECT/WHERE; build JS data dashboard.

### **Day 4 — Async programming + SQL**

Promises, async/await, fetch, HTTP methods; Java classes/constructors; SQL CRUD, joins, aggregates; DSA sliding window; API calls; Git commits.

### **Day 5 — DOM + REST fundamentals**

DOM/events/forms, JSON, REST principles/status codes; Java inheritance/interfaces; SQL GROUP BY/HAVING/index basics; DSA stack/queue; build CRUD UI against mock API.

### **Day 6 — Python + JavaScript problem solving**

Exceptions, files, JSON, virtual environments; JS error handling; Java exceptions/collections; DSA linked list; create small Python API client; document learning.

### **Day 7 — Week 1 checkpoint**

20 theory questions, 10 coding questions, 5 SQL questions, 1 debugging task, mini-project review, README cleanup, GitHub push; score yourself /100.

## **Week 2 — Full Stack Core**

Move into React, Node, MongoDB and Spring Boot.

### **Day 8 — React foundations**

Vite, JSX, components, props, state; Java collections/streams; MongoDB documents/CRUD; DSA binary search; build product-list UI.

### **Day 9 — React hooks + routing**

useState/useEffect, forms, React Router; Spring Boot project structure, DI; MongoDB indexes; DSA recursion; connect frontend to API.

### **Day 10 — Node + Express**

Node runtime, modules, Express routing/middleware; Java REST controller basics; SQL schema design; DSA trees; build CRUD backend.

### **Day 11 — Validation + error handling**

Express validation/error middleware; Spring DTO/validation/exception handling; Mongoose schemas; DSA BST; API testing in Postman.

### **Day 12 — Authentication fundamentals**

Password hashing, JWT, cookies/sessions, CORS; Spring Security basics; DB user schema; DSA heap/priority queue; implement auth.

### **Day 13 — TypeScript + frontend quality**

Types, interfaces, generics basics; React form validation and loading/error states; Java Optional/streams; DSA BFS/DFS; refactor MERN project.

### **Day 14 — Week 2 checkpoint**

Deploy frontend/backend prototype; 20 theory + 10 coding + 5 SQL + project viva; fix weakest two areas.

## **Week 3 — Production APIs**

Build backend like a junior production engineer.

### **Day 15 — Advanced REST APIs**

Pagination, filtering, sorting, versioning, consistent error schema; Spring pagination; DSA graphs; add API documentation.

### **Day 16 — Database engineering**

Transactions, isolation, indexes, query plans; Mongo aggregation; Java JPA/Hibernate relationships; DSA greedy; optimize two slow queries.

### **Day 17 — Redis + caching**

Cache-aside, TTL, invalidation, sessions; Spring Data Redis; Node Redis integration; DSA DP basics; cache product/search endpoint.

### **Day 18 — Testing**

Jest/Vitest concepts, API integration tests; JUnit + Mockito + Spring Boot tests; DSA mixed set; create CI test workflow.

### **Day 19 — Security**

OWASP basics, input validation, RBAC, rate limiting, secret management; secure JWT; DSA mixed medium problems; security review.

## Day 20 — Frontend production quality

TanStack Query concepts, optimistic updates, reusable components, accessibility, performance; connect robust API states; DSA timed set.

## Day 21 — Project milestone

Finish MERN e-commerce MVP: auth, products, search, cart, admin basics, tests, README, architecture diagram, deployment; weekly assessment.

# Week 4 — Microservices + DevOps

Learn distributed systems and production deployment.

## Day 22 — Architecture evolution

Monolith vs modular monolith vs microservices; service boundaries; API Gateway; Java service decomposition; DSA graphs; design order/payment services.

## Day 23 — Messaging + reliability

Queues, Kafka/RabbitMQ concepts, retries, timeouts, circuit breaker, idempotency; implement mock async notification; DSA mixed.

## Day 24 — Payment architecture

Order/payment lifecycle, sandbox integration, webhook verification, payment states, idempotency, refunds/reconciliation concepts; build mock payment service.

## Day 25 — Docker

Dockerfile, images, containers, volumes, networks, Compose; containerize MERN/Spring services; health checks; DSA timed practice.

## Day 26 — CI/CD

GitHub Actions, lint/test/build, environment secrets, deployment workflow; create CI for one frontend and one backend.

## Day 27 — Cloud + reverse proxy

Nginx, HTTPS, domains, cloud fundamentals, logs, health endpoints; deploy project; troubleshoot CORS/env/build issues.

## Day 28 — Monitoring + Week 4 review

Structured logs, metrics, uptime, alerts, basic Prometheus/Grafana concepts; document deployment; 30 interview questions + system design mock.

## Week 5 — AI/LLM Foundations

Transition from software engineering into production AI applications.

### Day 29 — AI/ML fundamentals

AI vs ML vs DL, supervised/unsupervised, train/validation/test, overfitting, metrics; Python NumPy/pandas/scikit-learn; implement baseline classifier.

### Day 30 — Neural nets + transformers

Neural network intuition, tokens, attention, transformer architecture; Python preprocessing; explain inference pipeline; DSA timed set.

### Day 31 — LLM fundamentals

Tokens, context, temperature, top-p, prompting, system/user messages, streaming; call an LLM API safely with env vars; build CLI assistant.

### Day 32 — Structured generation

JSON/schema output, validation with Pydantic, retries, prompt templates; FastAPI endpoint; build structured extractor.

### Day 33 — Prompt engineering

Zero-shot/few-shot, constraints, examples, prompt versioning, injection awareness; create prompt test set and regression checks.

### Day 34 — AI backend

FastAPI routing, async, auth, background tasks, streaming/WebSockets concepts; Dockerize AI service; connect React frontend.

### Day 35 — AI checkpoint

Build and deploy an LLM-powered assistant; measure latency/cost/errors; answer 20 AI questions + 10 Python problems.

## Week 6 — Embeddings + RAG

Build a real retrieval-augmented generation system.

### Day 36 — Embeddings

Embedding intuition, vectors, cosine similarity, semantic search; generate embeddings; visualize/query nearest neighbors conceptually; build semantic search.

### Day 37 — Vector databases

Chroma, FAISS concepts, pgvector concepts, metadata filtering, collections/indexes; store document chunks and retrieve them.

### Day 38 — Document ingestion

PDF/text loading, cleaning, chunking, metadata, overlap; build ingestion pipeline; test chunking quality.

### Day 39 — RAG pipeline

Query embedding → retrieval → context construction → LLM → answer; add citations; build FastAPI RAG endpoint.

### Day 40 — Advanced retrieval

Hybrid search, query rewriting, multi-query retrieval, reranking, parent-child retrieval concepts; compare retrieval variants.

### Day 41 — RAG evaluation

Golden dataset, relevance, faithfulness/groundedness, retrieval metrics, LLM-as-judge concepts; create 20 evaluation cases.

### Day 42 — Deploy RAG

React UI + auth + document upload + RAG; Docker + deployment; logs, token usage and latency tracking; weekly assessment.

## Week 7 — Agents + MCP

Build reliable agentic systems rather than simple chatbots.

### Day 43 — Agent fundamentals

Agent vs workflow, tools, tool schemas, state, routing; implement simple tool-calling assistant.

### Day 44 — LangChain

Models, prompts, retrievers, tools, output parsers, chains; reproduce a small RAG workflow and explain internals.

### Day 45 — LangGraph

State, nodes, edges, conditional routing, loops, persistence concepts; build research/workflow agent.

## Day 46 — Agent memory + human approval

Short/long-term memory concepts, checkpoints, human-in-the-loop, permission boundaries; add approval step for sensitive tool.

## Day 47 — MCP

MCP architecture, resources, tools, prompts, schemas, client/server; build small MCP server exposing safe tools.

## Day 48 — Agent security + evaluation

Prompt injection, tool abuse, excessive agency, output validation, rate limits; create agent test suite and failure cases.

## Day 49 — Agent deployment

FastAPI + React agent UI, streaming, Docker, deployment, logs/traces, cost/latency monitoring; weekly system-design + AI interview test.

# Week 8 — Capstone + Job Readiness

Combine everything into a portfolio-grade production system.

## Day 50 — Capstone architecture

AI e-commerce/business automation platform; requirements, HLD, LLD, ERD, APIs, service boundaries, threat model; create GitHub project.

## Day 51 — Core platform

Auth, users, products/orders, database, REST APIs, React dashboard; implement tests and CI.

## Day 52 — AI layer

Embeddings, semantic search, RAG knowledge base, recommendations, LLM assistant; add evaluation dataset.

## Day 53 — Agent + tools

LangGraph agent, tool calling, MCP integration, human approval, permission boundaries; failure handling.

## Day 54 — Payments + production

Sandbox payment flow, webhooks, idempotency, Redis caching, background jobs, notifications; Docker Compose and deployment.

## Day 55 — Observability + hardening

Logs, metrics, tracing concepts, health checks, security review, performance tests, bug fixing, documentation, architecture diagram.

## Day 56 — Final demo + interviews

Deploy final system; prepare 2-minute and 10-minute project explanations; DSA mock; Java/DBMS/OS/CN/API/AI/system-design mock interview; resume/GitHub review; calculate job-readiness score.

## 4. Project Roadmap

| Project                | Stack                                        | Core features                                               | Production target                           |
|------------------------|----------------------------------------------|-------------------------------------------------------------|---------------------------------------------|
| MERN E-Commerce        | React/TS, Node/Express, MongoDB, Redis       | Auth, catalog, search, cart, orders, admin, sandbox payment | Docker, CI/CD, deployment, logs             |
| Java Order System      | Java, Spring Boot, PostgreSQL, JPA, Security | Users, products, orders, pagination, validation             | JUnit/Mockito, Swagger, Docker              |
| Payment Microservices  | Spring/Node, Redis, queue, API gateway       | Order, payment, notification, webhooks, idempotency         | Compose, retries, health checks, monitoring |
| RAG Assistant          | React, FastAPI, embeddings, vector DB, LLM   | Upload, chunk, retrieve, citations, evaluation              | Docker, deployment, latency/cost metrics    |
| Agentic Assistant      | React, FastAPI, LangGraph, MCP               | Tools, state, approval, APIs, DB queries                    | Security, evaluation, tracing               |
| AI E-Commerce Capstone | Full stack + AI + microservices              | Recommendations, RAG, agent, payments, analytics            | CI/CD, monitoring, portfolio-ready          |

## 5. DSA Roadmap

**Daily target:** 3–5 quality problems, prioritizing pattern recognition over random volume. Use Java for interview implementations.

| Phase             | Topics                                                                | Target          |
|-------------------|-----------------------------------------------------------------------|-----------------|
| Foundation        | Arrays, strings, hash maps, two pointers, sliding window, prefix sums | 25–35 problems  |
| Linear structures | Linked list, stack, queue, monotonic stack                            | 20–25 problems  |
| Search/tree       | Binary search, recursion, trees, BST, heap                            | 25–30 problems  |
| Graphs            | BFS, DFS, topological sort, shortest-path concepts                    | 15–20 problems  |
| Optimization      | Greedy + DP basics                                                    | 15–20 problems  |
| Final             | Mixed timed sets + company-style questions                            | 8–10 timed sets |

## 6. System Design Roadmap

- Week 1: client/server, DNS, HTTP/HTTPS, latency, availability basics.
- Week 2: REST APIs, databases, caching, authentication, scaling basics.
- Week 3: load balancing, CDN, replication, indexing, queues, rate limiting.
- Week 4: microservices, API gateway, idempotency, retries, circuit breakers, payment architecture.
- Week 5–6: RAG architecture, vector retrieval, AI latency/cost, evaluation.
- Week 7: agent architecture, tool boundaries, state, human-in-the-loop, observability.
- Week 8: HLD/LLD mocks — URL shortener, e-commerce, payment system, chat, notification system, AI RAG platform.

## 7. AI Skill Progression

| Level            | Skills                                                               |
|------------------|----------------------------------------------------------------------|
| Foundation       | Python, ML concepts, metrics, tokenization, transformers             |
| LLM Apps         | Prompting, APIs, structured output, streaming, FastAPI               |
| Retrieval        | Embeddings, vector DB, chunking, semantic search, RAG                |
| Advanced RAG     | Hybrid retrieval, reranking, query rewriting, evaluation             |
| Agents           | Tools, LangGraph, state, memory, approval, failure handling          |
| Interoperability | MCP resources/tools/prompts, secure tool schemas                     |
| Production AI    | Evaluation, security, observability, latency, token cost, deployment |

## 8. GitHub Repository Strategy

- 01-mern-ecommerce — complete MERN project.
- 02-java-spring-order-service — Java/Spring Boot backend.
- 03-payment-microservices — payment/order/notification architecture.

- 04-rag-knowledge-assistant — RAG application.
- 05-agentic-ai-mcp-assistant — agent + tools + MCP.
- 06-ai-ecommerce-platform — final capstone.
- Every repository should contain: README, architecture diagram, setup, env.example, API docs, tests, screenshots, deployment URL, limitations, roadmap.

## 9. Daily Feedback System

| Score | Action                                                                                                |
|-------|-------------------------------------------------------------------------------------------------------|
| 0–5   | Stop and repair the gap: reread concept, rebuild from scratch, solve 2 extra problems, explain aloud. |
| 6–7   | Continue, but add one debugging or extension task before moving on.                                   |
| 8–10  | Continue and add an advanced feature or optimization; document the trade-off.                         |

At every 7-day checkpoint: review GitHub activity, deployed features, DSA accuracy, interview explanations, project architecture, and the weakest skill area.

## 10. Job-Readiness Score

| Area                                         | Weight |
|----------------------------------------------|--------|
| Programming + Java + JavaScript/TypeScript   | 15     |
| React + backend + APIs                       | 15     |
| Databases + Redis + microservices            | 10     |
| DevOps + deployment + monitoring             | 10     |
| DSA + core CS                                | 15     |
| System design                                | 10     |
| Python + ML + LLM                            | 10     |
| Embeddings + RAG + agents + MCP + evaluation | 10     |
| Projects + GitHub + interview communication  | 5      |

**Interpretation:** 0–40 Beginner • 41–60 Learning • 61–75 Junior-ready • 76–85 Strong fresher • 86–100 Highly competitive. Treat this as a self-assessment, not a hiring guarantee.

## 11. Final Definition of Done

- Can build a responsive React/TypeScript application from scratch.
- Can build secure REST APIs in Node/Express and Java/Spring Boot.
- Can design SQL and MongoDB schemas and explain indexing/transactions.
- Can implement JWT/RBAC, validation, errors, rate limiting and safe secret handling.
- Can explain and build a small microservice architecture with Redis and messaging concepts.
- Can integrate sandbox payments with webhook verification and idempotency.
- Can Dockerize and deploy applications and explain CI/CD.
- Can diagnose common production failures using logs, health checks and metrics.
- Can build an LLM application with structured output and streaming.
- Can create embeddings and semantic search with a vector database.
- Can build, evaluate and improve a RAG system with citations.
- Can build an agent with tools, state, approval and safe boundaries.
- Can build and explain an MCP server/tool integration.
- Can discuss AI evaluation, security, latency and cost.

- Can solve common DSA patterns in Java and explain complexity.
- Can design beginner/intermediate systems at HLD and LLD level.
- Has 5–6 documented projects, deployed demos, GitHub repositories and a final capstone suitable for interviews.

## 12. Recommended Daily Operating Routine

- Block 1 — 90 min: learn the day's concepts and take handwritten/typed notes.
- Block 2 — 90–120 min: code the concept without copying a tutorial.
- Block 3 — 120–180 min: build the project feature.
- Block 4 — 60 min: DSA in Java.
- Block 5 — 30 min: system design/interview questions.
- Block 6 — 20–30 min: Git commit, README/update tracker, self-score.
- Every Sunday/checkpoint: demo what you built, fix bugs, and revise weak concepts.

## Final outcome

If you complete the 56 days honestly, you should be able to demonstrate a broad fresher-level engineering profile spanning **MERN + Java/Spring Boot + APIs + databases + microservices + payments + Docker/CI/CD + deployment/monitoring + Python + LLMs + embeddings + RAG + agents + MCP + AI evaluation/security**. The strongest evidence will be working deployed projects and your ability to explain their architecture, trade-offs, failures and improvements.